Implementation Plan

Decision-Aware Model-Family Selection for Time Series Foundation Models

Algoverse AI Research · Target: AI4TS at AAAI-27

1 Summary

Six stages: pre-registration, premise gates, one expensive generation pass, a decision layer, four analysis experiments, and writing. **Predictions are generated once, cached to disk, and every later experiment is arithmetic on those files.** That is why one stage is heavy and the rest are nearly free.

Compute. The entire forecaster suite, all five checkpoints loaded at once, is about **499 million parameters, roughly 1 GB in half precision**. A free T4 has 16 GB, so nothing here is memory-bound and no stage needs an A100 to fit.
We train nothing, so this is inference, which the compute policy places on free tiers. No hosted language model is called at any point, so the **API budget requirement is zero**. The plan runs on free GPU tiers, with one A100 grant held in reserve for throughput.

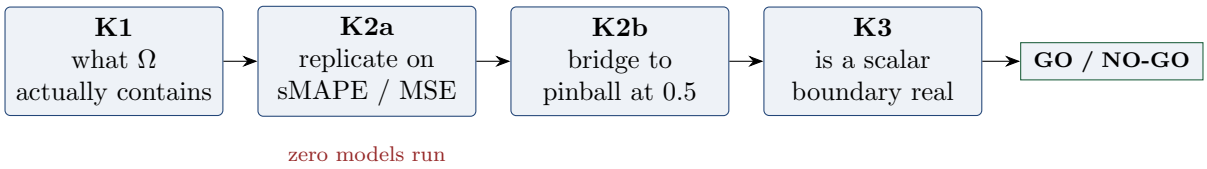
2 Stage 0: Pre-registration and data preparation (Week 1, first half)

Before anything runs: write and timestamp the grid, namely the exact dataset list, the forecast horizons (proposed: $h \in \{1, 24, 168\}$ for hourly data, to be confirmed with the proposal before freezing), and the operational quantiles $\tau \in \{0.01, 0.05, 0.09, 0.25, 0.40, 0.50, 0.60, 0.75, 0.91, 0.95, 0.99\}$. Fix the change-point detector. Fix the development and held-out split, with M5 and PJM held out entirely.

Apply the length floor. The source paper recommends Ω only for series longer than 1000 timesteps, because FFT estimates become unstable below that. Every dataset configuration whose usable training history falls below the floor is excluded, and the exclusion list is recorded and reported in the paper. This happens before the grid is frozen.

3 Stage 1: Premise gates (Week 1, second half)

These are the proposal's E0 gates, using the same names. They answer whether the project is viable.



K1. What does Ω actually contain? Implement the statistic and confirm from the published definition that it is determined by second-order and spectral information, which is the assumption the matched-Ω construction in E3 depends on. Validate the implementation against synthetic signals of known structure: pure sine ≈ 0.87, daily plus weekly cycle ≈ 0.79, moderate noise ≈ 0.32, white noise ≈ 0.06. Decide and document whether Ω is computed on levels or first differences, since a random walk scores ≈ 0.74 on levels despite being unpredictable.

K2a. Replicate on the source paper's own metrics. The rule we build on was validated on GIFT-Eval, which publishes per-model results for over 100 models across 97 dataset configurations. Compute Ω on training splits, join to the published sMAPE and MSE results, and check whether the pattern is there. **No forecasting model is run and no metric is changed.** One day of one person, zero GPU.

K2b. Bridge to the decision metric. Only after K2a passes, check whether the relationship survives the move to pinball loss at $\tau = 0.5$. Kept separate because pinball at 0.5 is proportional to mean absolute error and is not a metric the original result was established on. Collapsing the two would leave a failure ambiguous between a false premise, a broken pipeline, and a metric change. A K2b failure is informative in its own right.

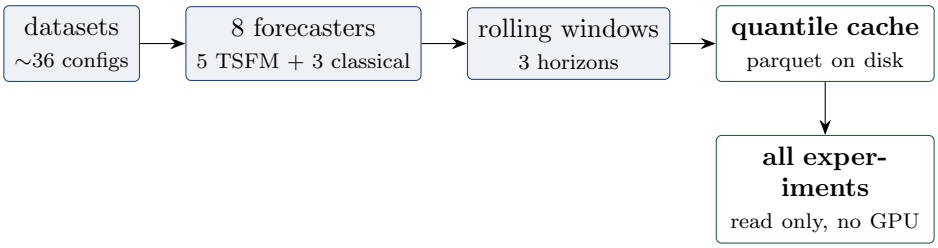
K3. Is a scalar boundary meaningful? A fitted curve always returns a boundary. Before fitting, count sign reversals in Δ, compare a monotone fit against an unconstrained one, and measure how much oracle value the best single threshold captures. If it captures little, we stop fitting $\Omega^*(\tau)$ and the paper reframes around insufficiency.

Gate. If K2a fails we change direction in week one rather than week eight. A 2026 paper reports descriptor-based selection correct in only a small fraction of cases, so this is a live risk. The pivot is already in the proposal: we ask instead whether decision-aligned selection is more reliable than error-based selection.

4 Stage 2: Generate all predictions (Weeks 2 and 3)

The only expensive stage, run exactly once.

Every forecaster runs across every dataset and every rolling window, and we save the **full predictive distribution**, a grid of quantiles from the 1st to the 99th percentile, not the point forecast. The decision depends on where you sit in that distribution, and that cannot be recovered later from an average.



Once the cache is written, the τ sweep, all four experiments, every re-analysis, and every figure regeneration cost nothing. Analysis will be repeated dozens of times; generation happens twice at most.

Cache contract. Every row is keyed by a content hash of `(dataset, config, window_start, model, horizon, seed)`. Writes are idempotent, so a rerun after an interrupted session resumes rather than duplicating. Store as partitioned Parquet, and sync to persistent storage immediately, because the A100 machines delete everything at expiry.

5 Stage 3: Decision layer and baselines (Week 4)

Everything here reads the cache. No model runs.

Freeze the within-family rule first. Within each family, take the candidate with the lowest mean pinball-τ on the development datasets at $\tau = 0.5$, and freeze it before any held-out evaluation. Development and held-out sets are disjoint, so no held-out information enters the choice, and the same rule applies to both families. Record the chosen representatives in the manifest. A robustness pass repeats the headline comparison using a fixed representative per family declared in advance.

Two decision tasks, each paired with data that faces it. M5 retail demand feeds a newsvendor ordering decision under asymmetric overage and underage costs. PJM electricity feeds reserve procurement under asymmetric shortage and excess penalties. Electricity cannot be inventoried, so pairing it with a stock-ordering decision would be spotted immediately. For each, take the cached quantile grid, extract the decision-optimal percentile at τ, compute realized cost, and sweep all eleven pre-registered τ values. Minutes of CPU per sweep.

Compute all seven baselines. These are the comparators of Section 6.1 of the proposal. Each closes a necessary question, and each is a query over the same cache.

Note on the word "baseline", which means two different things in this project. A *baseline model* is a cheap classical forecaster, one of the two candidate families. A *baseline selector* is one of the seven rows below, against which D_Ω is measured. "Always classical" is a baseline selector that always picks a baseline model. We use "baseline model" and "baseline selector" wherever the sense is not obvious from context.

Comparator	What it establishes	Cost
Always classical	Cheap fixed-family floor	free
Always TSFM	Expensive fixed-family floor	free
Published static Ω rule	Prior method being tested and corrected	free
Pinball-τ validation	Exact decision-aligned selector; the approximation target	free
CRPS validation	Generic probabilistic selector: does decision alignment add anything	free
D_Ω	Our proposed cheap decision-aware selector	free
Hindsight family oracle	Ceiling, and the denominator for normalized regret	free

CRPS stays in the main table. The likeliest reviewer objection is that a well-calibrated predictive distribution already suffices once the τ-quantile is extracted, and CRPS-based selection tests that directly. It is also the quantile integral of pinball loss, so the comparison is exact, and it costs nothing from cached forecasts.

6 Stage 4: The four experiments (Weeks 5 to 7)

All four read the cache. None runs a model.

E1. Where does the boundary sit as costs get lopsided? Sweep the operational quantile on *both* sides of 0.5: $\tau \in \{0.01, 0.05, 0.09, 0.25, 0.40, 0.50, 0.60, 0.75, 0.91, 0.95, 0.99\}$. Both sides matter because the candidate boundary is written in $|\tau - 0.5|$ and therefore assumes symmetry; sweeping both tests that rather than asserting it. Cost: minutes of CPU.

E2. Does it transfer? Fit the selector on development collections, freeze every coefficient, then evaluate on M5 retail and PJM electricity, which are never touched during fitting. This is the honesty test, since anything works on data it was tuned on.

E3. Matched-Ω sufficiency test. Construct synthetic series that are identical under Ω but different underneath, holding the spectrum fixed while changing innovation skew or tail weight. If they need different families, no selector using only Ω could separate them, which is a statement about the descriptor rather than about our fitted rule. Cheap in compute, demanding in care.

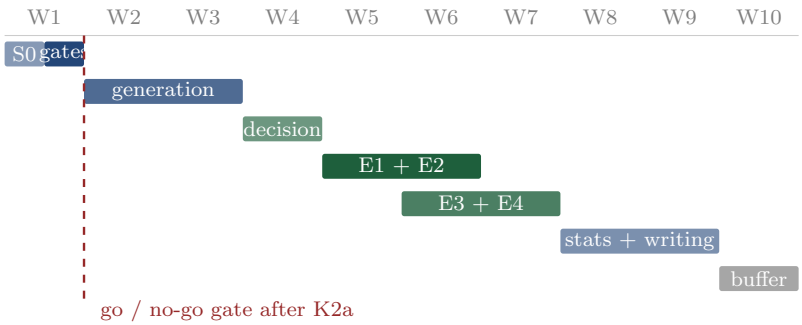
E4. Selective validation. Let the selector answer classical, TSFM, or defer to full validation, then plot validations avoided against excess decision regret. The output is a deployment figure: skip $X\%$ of the work for $Y\%$ worse decisions.

7 Stage 5: Statistics and writing (Weeks 8 to 10)

Rolling windows on the same series overlap heavily and are not independent, so treating them as independent trials gives confidence intervals that are far too narrow. Use moving block bootstrap at the series and domain level. Apply forecast-comparison tests only to the headline claims, with multiple-comparison correction there rather than across every exploratory number. Report costs in money as well as normalized regret for M5 and PJM.

Then paper, figures, and code release, with week 10 held as buffer.

Timeline



8 Models

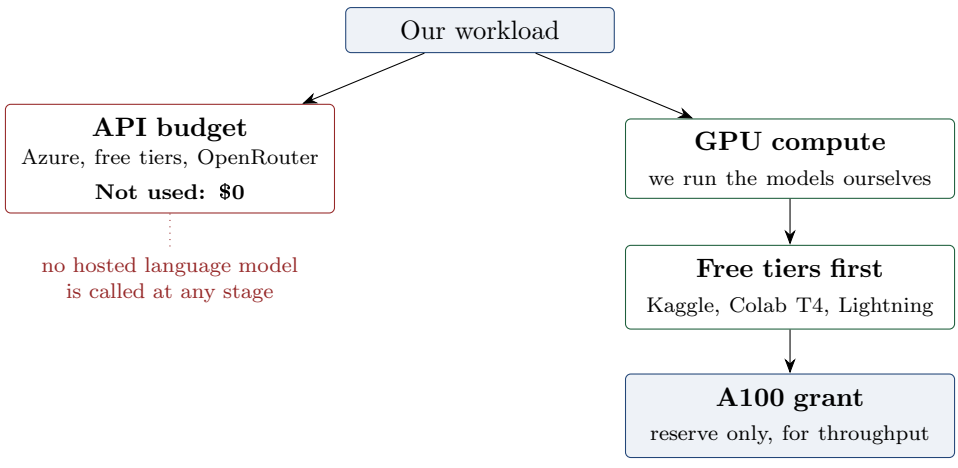
Chosen for three properties: a consistent quantile interface across both families, small enough to run on free hardware, and comparability with the published result we replicate.

Family	Model	Params	Why this one
TSFM	Chronos-Bolt-base	205 M	**Primary.** Emits quantiles directly, so no sampling loop
TSFM	Chronos-Bolt-small	48 M	Size-comparison point within one architecture
TSFM	TimesFM 2.5	231 M	Independent architecture, native quantile head
TSFM	Moirai-1.1-R-small	14 M	Probabilistic output, different pretraining corpus
TSFM	Tiny Time Mixers r2	0.8 M	Extreme-small control: does scale matter at all
Classical	AutoARIMA	n/a	Native prediction intervals
Classical	AutoETS	n/a	Native prediction intervals
Classical	SeasonalNaive	n/a	Floor, with empirical residual quantiles

Chronos-Bolt rather than Chronos-T5. The original Chronos generates by sampling, so a quantile grid needs 20 or more forward passes per window. Chronos-Bolt emits quantiles in one pass, roughly a twentyfold reduction for the same output. Chronos-T5-small is retained only if K2a replication requires matching the original roster.
Baseline models must produce distributions too. A probabilistic foundation model compared against a point-forecast ARIMA is not a fair experiment, and a reviewer will say so.

9 Compute strategy

9.1 Which budget applies



9.2 Estimated requirement

With a budget of 25 to 30 GPU-hours available, we spend it on *grid size* rather than bigger hardware, because bigger hardware buys us nothing here and a broader grid directly strengthens the paper.

Workload	**Est. GPU-hours**	**Where it runs**
K1 to K3 gates	<1 (mostly CPU)	Colab free T4
TSFM generation, Chronos-Bolt path	5 to 8	Kaggle T4 ×2
TSFM generation, other three models	10 to 15	Kaggle T4 ×2
Classical generation	0 (CPU, ∼25 CPU-hrs)	any machine, parallelized
Stage 3 decision layer and baselines	0	laptop or Colab CPU
E1 to E4 analysis	0	laptop or Colab CPU
Reruns, debugging, contingency	6	free tier
Total	**22 to 30 GPU-hours**	

What the extra budget buys. The earlier 12 to 20 hour estimate covered roughly 36 dataset configurations. Thirty hours covers **every GIFT-Eval configuration clearing the 1000-timestep floor**, about 70 of the 97, plus more rolling windows each.
Broader coverage makes the E2 transfer result harder to dismiss as dataset selection. More windows tighten the block-bootstrap intervals, which matters because overlapping windows already cost us effective sample size. Neither needs a larger GPU; both are more passes of the same small models.
Free capacity. Kaggle gives 30 GPU-hours per week per person, so four students have 120 per week against a requirement of 22 to 30. With Colab's free T4, Lightning's monthly credits, and Modal and Beam credits on top, the free tier covers this several times over.

9.3 On Google Colab Pro

Not reimbursed: the compute policy replaced Colab Pro reimbursement with A100 access, so any purchase is out of pocket.

Its value here is session length, not GPU power. Our suite is about 1 GB, so we are neither memory- nor compute-bound. Kaggle's 12-hour session cap is the only free-tier constraint that could bite during generation, and Colab Pro lifts it.

We design around it instead: generation is chunked by dataset configuration so no unit of work approaches 12 hours. If chunking proves awkward in week 2, one month is a reasonable purchase against a measured problem.

If bought, spend units on the T4, roughly 2 units per hour and 50 hours per 100 units, not the A100 at 12 to 15 units per hour.

9.4 If an A100 grant is used

The A100 machines are time-boxed and destructive, so they need their own protocol.

Constraint	What we do about it
Countdown starts at approval, not first login	Submit the request only when the code is already validated and the dataset is staged. Never request speculatively.
24-hour grant gives ∼20 usable hours	Plan for 16, not 20. Reserve the last two hours for syncing results off the machine.
Everything is deleted at expiry	Write the quantile cache to external storage continuously, not at the end. Sync after every dataset completes.
Idle ∼1 hour is reclaimed	Never hold the machine open while thinking. Stage work, run, sync, log off.
One machine per team	One nominated operator per grant. Others do not request in parallel.
May share a card with another team	Cap memory use well below 40 GB. Our suite needs about 1 GB, so this is not a constraint for us.
L4 backup, 24 GB	Fully sufficient for every model here. Accept the L4 without complaint if the A100 queue is full.

10 Tracking, monitoring, and optimization

10.1 The GPU-hour ledger

One shared sheet, one row per session, filled in at the end of each session rather than reconstructed later.

Date	Operator	Platform	Hours	Mean GPU util	What was produced
example: 12 Sep	A.	Kaggle T4×2	2.4	71%	Chronos-Bolt, 9 dev configs cached

Weekly, one person totals the ledger and compares it against the 12 to 20 hour estimate above. If we cross 20 hours before generation is complete, that is a signal to investigate throughput rather than to request more hardware.

10.2 Utilization, not uptime

A GPU at 5% utilization for four hours is worse value than one at 80% for one hour, and low utilization usually means the bottleneck is elsewhere.

- Run `nvidia-smi` sampling in the background of every session and log mean utilization into the ledger.
- **Target above 60%.** Below 30% means the GPU is starved, which for us will almost always be data loading or per-window Python overhead, not model size.
- The fix is batching across windows rather than looping one at a time. Our windows are independent, so they batch trivially.
- Pre-tokenize and pre-slice all windows into arrays before the run starts. Do not construct them inside the loop.

10.3 Cost-control rules we adopt

1. **Free tier first, always.** Every line of code is validated on a free T4 with a tiny subset before any larger machine is touched.
2. **Never leave a session idle.** Stop Kaggle sessions from the Active Events panel rather than closing the tab, since the meter runs while the session is open.
3. **Estimate before running.** Multiply expected windows by measured seconds-per-batch before launching a full pass. If the estimate exceeds budget, reduce the grid rather than requesting hardware.
4. **Checkpoint and sync continuously**, so an interruption costs minutes rather than a session.
5. **One operator per grant**, to avoid duplicate requests and split attention.

10.4 Instrumentation built into the code

- Every generation run writes a manifest: model, checkpoint hash, dataset, window range, wall-clock, GPU seconds, and rows produced. This makes the compute table in the paper a query

rather than a recollection.

- A resume check on startup: read the cache, skip anything already present. Reruns should cost only what is missing.
- Fixed seeds recorded per row, so any result can be regenerated exactly.
- A dry-run flag that reports projected cost without executing, used before every full pass.

11 Division of work

Role	Weeks	Owns
Generation lead	2 to 3 heavy	Cache architecture, batching, manifests, the GPU ledger. Everyone is blocked on this, so it starts first.
Decision layer	3 to 5	Turning a quantile grid into an action and a cost, both decision tasks, the τ sweep.
Mechanism owner	4 to 7	E3 matched-Ω construction and its numerical verification. Most statistically demanding piece; needs its own owner and an early start.
Analysis and paper	throughout	Experimental design, figures, statistics, writing, and keeping claims honest against what the data supports.

12 Risks and what we do about them

Risk	Response
C2 replication fails	Tested in week one at zero GPU cost. Project reframes around whether decision-aligned selection is more reliable than error-based selection, which is arguably the stronger paper.
Ω misreads integrated series	Random walk scores ≈ 0.74. Levels versus first differences is decided and documented on day one, and both are reported.
E3 overruns	Most likely schedule risk. Dedicated owner, started during stage 2 rather than stage 3.
Classical baseline models are slow	AutoARIMA over tens of thousands of fits is the CPU bottleneck, not the GPU. Parallelize across cores and cache fits; this is not a GPU problem and must not be solved by requesting a GPU.
A100 grant expires with unsynced work	Continuous sync, and the last two hours of any grant reserved for retrieval.
Free-tier session limits	Kaggle sessions cap at 12 hours. Generation is chunked by dataset so no single unit of work exceeds a session.

13 What we ask the program for

- **API access: none.** No hosted language model is called at any stage.
- **AWS $200 credit: not requested for now.** Free tiers cover the estimate with an order of magnitude to spare. We would rather leave the credit unclaimed than spend it to no purpose.
- **A100: one 24-hour grant, held in reserve.** Requested only if week-2 throughput measurement shows the consolidated pass will not fit free-tier sessions, and only once the code is validated and data staged so the countdown is not wasted.
- **Everything else runs on per-student free tiers**, which each teammate registers for individually as the policy requires.